

Prepared by: Bumjun Ko
Academic Advisor: Dr. Erdogan Dogdu
Proposed Course: Senior Design Project (CS-4399-D10)
Project Owner: Center for International Studies, CIS
Proposed Development Period: June 2, 2026 – July 3, 2026
Project Type: Full-Stack Web Application

Project Proposal and Technical Development Plan

1. Executive Summary

The proposed project is the development of a web-based student blogging platform for the English Language Learners' Institute (ELLI), operated through the Center for International Studies (CIS) at Angelo State University. The purpose of the platform is to allow ELLI students to write, submit, and share blog posts about their English learning experiences, cultural activities, trips, campus life, and program-related events.

The project will be developed as a full-stack web application. It will include student registration and login, blog post submission, featured image upload, consent collection, student dashboard functionality, CIS Staff administrative review, an approval and publishing workflow, and a public-facing blog page that displays only approved and published posts.

The platform will be owned by CIS. The initial development will be completed by Bumjun Ko as part of the Senior Design Project course (CS-4399-010). After the initial prototype is developed, the application, documentation, repository, deployment instructions, and maintenance guidance will be prepared so that the ELLI Coordinator or CIS Staff can manage and maintain the system. The project will be designed as a functional MVP prototype during the project period, with future production hosting to be reviewed by ASU IT/Web Services or, if approved by CIS, through a low-cost cloud hosting option.

The primary goal of the Senior Design Project is not to build a large-scale commercial CMS, but to deliver a complete, working, secure, documented, and demonstrable MVP that supports the core workflow: Student creates an account → student submits a blog post with image and consent → CIS Staff reviews the submission → CIS Staff approves and publishes → approved and published post appears publicly.

This project is appropriate for a Computer Science Senior Design Project because it involves full-stack web development, database design, user authentication, role-based authorization, image storage, content moderation workflow, privacy and consent considerations, deployment planning, documentation, and real-world stakeholder requirements.

2. Project Background

The Center for International Studies (CIS) at Angelo State University supports international

students, language learners, cultural programming, and international engagement across the university community. As part of CIS, the English Language Learners' Institute (ELLI) provides a non-credit English language program for non-native English speakers. The program helps students develop English communication skills through reading, writing, listening, and speaking instruction. In addition, ELLI students practice pronunciation, vocabulary development, and grammar skills as they improve their ability to communicate in academic, cultural, and everyday settings.

Beyond language instruction, ELLI also provides students with opportunities to participate in cultural activities, campus experiences, trips, and program-related events. These experiences are valuable because they allow students to apply English in real-world contexts, engage with the Angelo State University community, and reflect on their personal and academic growth. However, student experiences from ELLI activities may currently be difficult to collect, organize, review, and present in a consistent and structured way.

A web-based student blogging platform would address this need by allowing ELLI participants to document their experiences in their own words while giving CIS a controlled process for reviewing and approving content before it becomes public. Through the platform, students would be able to write blog posts about their English learning experiences, cultural activities, travel, campus life, and program-related events. At the same time, CIS would be able to manage submitted content, review posts for appropriateness, confirm required consent, and publish only approved materials.

The platform would serve two related purposes. First, it would provide ELLI students with a meaningful opportunity to reflect on their learning, cultural experiences, trips, and campus involvement. Second, it would provide CIS with approved student-generated content that can help showcase the ELLI program to future students, campus partners, and public visitors.

Because the project involves student-created content, student names, images, and potentially program-related experiences, the platform must include a careful approval process, privacy consent, image-use confirmation, and staff-controlled publishing. These requirements make the project not only a web development task, but also a practical system that must consider real stakeholder needs, responsible content management, and student privacy.

3. Problem Statement

CIS needs a structured and controlled way to collect and publish ELLI student stories. Without a dedicated platform, student experiences may be collected through informal methods such as email, shared folders, documents, or manual web updates. These methods can create several challenges:

1. Student submissions may not follow a consistent format.
2. Images may be difficult to organize and associate with the correct post.
3. CIS Staff may not have a central dashboard for reviewing submissions.
4. There may be no clear status workflow showing whether a post is drafted, submitted, approved, rejected, or published.

5. Student consent and photo-use confirmation may not be consistently captured.
6. Published content may require manual coordination with web staff or other systems.
7. Future Student Assistants may have difficulty continuing the project without documentation.

The proposed platform addresses these challenges by creating a centralized web application where students submit content, CIS Staff reviews and approves it, and only approved and published posts appear publicly.

4. Project Purpose

The purpose of this project is to develop an MVP prototype of the ELLI Student Blogging Platform, a web-based system that supports student blog submissions and CIS Staff content approval.

The project will focus on building a practical application that can be demonstrated to CIS, Dr. Dogdu, and potentially ASU IT/Web Services. It will also include documentation so the project can be transferred to CIS or an ASU-managed account in the future.

The project is intended to be both an academic learning experience and a real-world software development project requested by CIS.

5. Project Ownership and Stakeholders

5.1 Project Owner

The official project owner will be:

Center for International Studies, CIS

CIS will be responsible for the purpose, content policy, administrative use, and long-term ownership of the application.

5.2 Developer

The initial developer will be:

Bumjun Ko

Student Assistant, Center for International Studies

Senior Design Project student developer

The developer will be responsible for implementing the MVP prototype, preparing documentation, and creating a handoff structure for future maintenance.

5.3 Academic Advisor

The academic advisor will be:

Dr. Erdogan Dogdu

The advisor will provide academic guidance, project evaluation, and feedback regarding whether the project satisfies the requirements of the Summer I Computer Science project course.

5.4 Administrative Users

Administrative users will be:

CIS Staff

CIS Staff will review submitted posts, request revisions, approve posts, reject posts, publish approved posts, archive posts, and respond to deletion requests.

5.5 Student Users

Student users will be:

ELLI participants with valid @angelo.edu email addresses

Students will be able to register, log in, submit blog posts, upload a featured image, provide required consent, view submission status, and edit posts subject to staff review.

5.6 Public Users

Public users will be:

Visitors to the public blog page

Public users will only be able to view blog posts that have been approved and published by CIS Staff. They will not be able to comment, create accounts, or submit content.

5.7 ASU IT/Web Services

ASU IT/Web Services may be involved in future production review, hosting review, domain review, accessibility review, security review, and possible integration with ASU systems. ASU Web Services supports software used for several Angelo State web and mobile applications, including angelo.edu and RamPort, and also works on user interface design, accessibility compliance, and training users on supported tools.

6. Project Ownership and Stakeholders

The project has the following major goals.

6.1 Functional Goal

Develop a working web application where ELLI students can submit blog posts and ELLI Coordinator can approve and publish them.

6.2 Academic Goal

Complete a full-stack web development project suitable for a Senior Design project course.

6.3 Operational Goal

Create a prototype that CIS can evaluate for future use and possible ASU IT/Web Services review.

6.4 Documentation Goal

Prepare documentation that allows CIS or a future Student Assistant to maintain the project.

6.5 Security and Privacy Goal

Avoid direct password storage, restrict student registration to @angelo.edu email addresses, implement role-based access control, collect consent, and ensure that only approved and published posts are publicly visible.

6.6 Maintainability Goal

Use a modern but manageable technology stack, organize the code clearly, document setup and deployment, and avoid unnecessary complexity in the MVP.

7. Project Objectives

By July 3, 2026, the project should accomplish the following:

1. Build a responsive web application using Next.js, TypeScript, Tailwind CSS, and Supabase.
2. Implement student registration and login using @angelo.edu email addresses.
3. Collect privacy and platform-use consent during account registration.
4. Allow students to create, edit, and submit blog posts.
5. Allow students to upload one featured image per blog post.
6. Require photo-use and public-posting consent every time a student submits a post.
7. Provide a student dashboard showing the student's own posts and statuses.
8. Provide a CIS Staff admin dashboard.
9. Allow CIS Staff to review submitted posts.
10. Allow CIS Staff to approve, reject, request revision, publish, or archive posts.
11. Ensure that public visitors can only view posts that are both approved and published.
12. Implement basic security controls using Supabase Auth, role-based authorization, and Row Level Security policies.
13. Prepare backup and maintenance recommendations.
14. Deploy the prototype for demonstration.
15. Write a README, setup guide, admin user guide, deployment guide, and maintenance handoff document.
16. Prepare final project report and demo presentation.
- 17.

8. Proposed Technology Stack

8.1 Frontend and Full-Stack Framework

The application will use:

Next.js with TypeScript

Next.js is a React framework for building full-stack web applications and provides routing, rendering, optimization, and application structure.

TypeScript will be used to improve code reliability, reduce type-related bugs, and make the project easier to maintain.

8.2 Styling

The application will use:

Tailwind CSS

Tailwind CSS will allow fast development of responsive user interfaces while keeping styling consistent.

8.3 Backend and Database

The application will use:

Supabase PostgreSQL

Supabase provides a Postgres database, Authentication, APIs, Storage, and other backend tools.

The database will store user profiles, blog posts, review statuses, consent records, deletion requests, and status history.

8.4 Authentication

The application will use:

Supabase Auth

Supabase Auth will handle registration, login, logout, session management, and password reset. Supabase Auth stores only hashed passwords using bcrypt with a randomly generated salt; the application itself will not store plaintext passwords.

Future production versions may explore ASU SSO if supported and approved by ASU IT.

8.5 File Storage

The application will use:

Supabase Storage

Supabase Storage will store uploaded blog images. Supabase Storage supports file storage with access controls, and upload limits can be configured depending on the plan.

For the MVP, each post will support one featured image. Multiple-image galleries will be considered a future enhancement.

8.6 Authorization

The application will use:

Role-based authorization and Supabase Row Level Security, RLS

Supabase RLS allows developers to implement access rules directly in Postgres so that users can only access the rows they are authorized to access.

The project will use at least two roles:

1. student
2. admin

Public visitors will only have read access to approved and published posts.

8.7 Deployment

The prototype may be deployed using:

Vercel + Supabase

Vercel provides simple deployment for Next.js applications. Vercel Pro is currently listed at \$20/month plus additional usage, and the Pro plan includes one deploying team seat and monthly usage credit.

For production, the preferred official path is to review hosting and integration possibilities with ASU IT/Web Services.

8.8 Version Control

The project will use:

GitHub

The repository will include source code, database migration files, documentation, setup instructions, and handoff materials. The repository should eventually be transferable to a CIS or ASU-managed account.

9. Hosting and Deployment Plan

The project will distinguish between three stages:

1. Development prototype
2. CIS review/pilot
3. Possible production deployment

9.1 Development Prototype

During Summer I, the application will be developed and deployed as a prototype. The recommended development deployment option is:

Vercel Free or Pro Trial + Supabase Free

This stage is for academic demonstration, testing, and feedback. No real sensitive student data should be used during early development.

9.2 CIS Review/Pilot

After the MVP is complete, CIS may test the system internally using sample data or limited pilot data. At this stage, the project may remain on Vercel and Supabase for evaluation.

9.3 Production Review

Before real production use, the project should be reviewed with ASU IT/Web Services.

Questions for ASU IT/Web Services may include:

1. Can the application be hosted on ASU infrastructure?
2. Can the application be linked from an official CIS or ELLI page?
3. Can the app use an ASU domain or subdomain?
4. Can the app use ASU SSO?
5. What data privacy and security review is required?
6. What accessibility requirements must be met?
7. What backup and retention policy is required?

8. Who will be responsible for long-term maintenance?

ASU Web Services provides a “Submit a Web Request” option and lists web.services@angelo.edu as a contact for Web Services.

9.4 Cloud Hosting Cost Estimate

If ASU IT hosting is not immediately available, a low-cost cloud option may be considered.

A likely paid cloud option is:

Service	Plan	Estimated Monthly Cost
Vercel	Pro	\$20/month plus additional usage
Supabase	Pro	
Estimated Base Total	-	Approximately \$45/month

Supabase Pro is currently listed with included resources such as 100,000 monthly active users, 8 GB database disk size, 100 GB file storage, 250 GB egress, and 250 GB cached egress.

Because ELLI is expected to have around 10 student users per semester, the project’s expected usage should be very small. However, the final cost may vary due to taxes, additional team seats, paid add-ons, or usage overages.

9.5 Free Tier Feasibility

For the Summer I prototype, free-tier usage should likely be sufficient. Supabase Free currently includes 50,000 monthly active users, 500 MB database size, 5 GB egress, 5 GB cached egress, and 1 GB file storage.

The free tier should be used only for prototype/demo purposes. For real production use, CIS should evaluate whether a paid plan or ASU-managed hosting is more appropriate.

10. Target Users

10.1 Student Users

Student users are ELLI participants who have valid @angelo.edu email addresses.

Student users need to:

1. Register and log in.
2. Accept privacy and platform-use consent.
3. Create blog post drafts.
4. Upload one featured image.
5. Confirm image-use permission.
6. Submit posts for review.
7. View the review status of their posts.
8. Edit submitted or published posts, knowing that edits require staff approval before becoming public.
9. Request deletion of published posts if needed.

10.2 CIS Staff Admin Users

CIS Staff admin users need to:

1. Log in securely.
2. View all submitted posts.
3. Review post text and uploaded images.
4. Check whether the student accepted consent.
5. Request revisions.
6. Approve posts.
7. Publish approved posts.
8. Reject posts.
9. Archive posts.
10. Respond to deletion requests.
11. Manage content according to CIS policy.

10.3 Public Visitors

Public visitors need to:

1. View the ELLI blog homepage.
2. Browse approved and published posts.
3. Open individual post pages.
4. See student stories and images that have been approved by CIS.
5. Use the site on mobile and desktop devices.

Public visitors will not be able to:

1. Register.
2. Submit content.
3. Comment.
4. View pending posts.
5. View rejected posts.
6. View student dashboard or admin pages.

11. Scope of the Summer I MVP

The Summer I MVP will focus on the core workflow needed for a functional blogging platform.

11.1 Must-Have Features

The following features must be completed by July 3, 2026.

Student Authentication

1. Student registration.
2. Student login.
3. Student logout.
4. @angelo.edu email restriction.
5. Password reset if supported within time.
6. Privacy consent during signup.

Student Dashboard

1. View all posts created by the logged-in student.
2. See each post's current status.
3. Create a new post.
4. Edit own posts.
5. Submit posts for review.

Blog Post Submission

1. Title field.
2. Main content field.
3. Optional category/activity field.
4. Featured image upload.
5. Image alt text field.
6. Photo-use consent checkbox.
7. Public posting consent checkbox.
8. Submit button.
9. Basic validation.

Image Upload

1. One featured image per post.
2. File type restriction: .jpg, .jpeg, .png, .webp.
3. Recommended file size limit: 5 MB per image for MVP.
4. Storage in Supabase Storage.

5. Image preview before submission if possible.

Admin Dashboard

1. Admin-only protected route.
2. View all submitted posts.
3. Filter or visually distinguish post statuses.
4. Open a post review page.
5. View title, content, image, author, consent status, and submission date.

Admin Review Workflow

1. Mark as under review.
2. Request revision.
3. Approve.
4. Publish.
5. Reject.
6. Archive.
7. Add admin note.

Public Blog

1. Blog list page.
2. Blog detail page.
3. Show only posts where:

`review_status = approved`
`AND`
`is_published = true`

4. Responsive layout.
5. No comments.

Documentation

1. README.
2. Setup guide.
3. Deployment guide.
4. Admin user guide.
5. Maintenance handoff guide.
6. Database schema documentation.
7. Known limitations and future enhancements.

12. Out-of-Scope Features for Summer I

The following features will not be included in the Summer I MVP unless all required features are completed early.

1. ASU SSO integration.
2. Comments.
3. Likes or reactions.
4. Multiple image galleries.
5. Rich text editor.
6. Email notifications.
7. AI writing assistant.
8. Automatic translation.
9. Advanced analytics.
10. Full CMS replacement.
11. Complex multi-level admin roles.
12. Full version history for post revisions.
13. Automated rejected-post deletion using scheduled jobs.
14. Advanced image moderation.
15. Integration with LiveWhale or RamPort.
16. Payment or billing features.
17. Mobile app version.

These may be considered future enhancements.

13. User Roles and Permissions

13.1 Public Visitor

Public visitors can:

1. View the homepage.
2. View public blog list.
3. View public blog detail pages.

Public visitors cannot:

1. View pending posts.
2. View rejected posts.
3. Submit posts.
4. Upload images.
5. Access dashboard pages.

6. Access admin pages.

13.2 Student

Students can:

1. Register with an @angelo.edu email.
2. Log in and log out.
3. View their own dashboard.
4. Create posts.
5. Upload one featured image.
6. Submit posts for review.
7. View their own post statuses.
8. Edit their own posts.
9. Request deletion of their own published posts.

Students cannot:

1. Approve posts.
2. Publish posts.
3. View other students' unpublished posts.
4. Access admin pages.
5. Directly make edited content public without staff approval.

13.3 Admin

Admins are ELLI Coordinator & CIS Staff.

Admins can:

1. View all posts.
2. Review submissions.
3. Change post review status.
4. Approve posts.
5. Publish posts.
6. Reject posts.
7. Request revisions.
8. Archive posts.
9. View deletion requests.
10. Resolve deletion requests.
11. Manage public visibility.

Admins should not directly see student passwords. Password management is handled by Supabase Auth.

14. Functional Requirements

FR-01: Student Registration

The system shall allow students to register with an email address.

Acceptance criteria:

1. The email must end with @angelo.edu.
2. The user must provide full name.
3. The user must accept privacy and platform-use consent.
4. The system must create a student profile.
5. The system must assign the default role student.

FR-02: Student Login

The system shall allow registered students to log in.

Acceptance criteria:

1. The student can log in with email and password.
2. The system creates a session.
3. The student is redirected to the student dashboard.
4. Invalid credentials show an error message.
5. Logged-in students can log out.

FR-03: Privacy Consent at Signup

The system shall collect privacy consent during account creation.

Acceptance criteria:

1. Signup cannot be completed unless the checkbox is selected.
2. The database stores the consent status.
3. The database stores the consent timestamp.
4. The database stores a consent version identifier.

FR-04: Student Dashboard

The system shall provide a dashboard for students.

Acceptance criteria:

1. The student can see a list of their own posts.
2. Each post displays title, status, last updated date, and public visibility.
3. The student can create a new post.
4. The student can edit their own post.
5. The student cannot see other students' unpublished posts.

FR-05: Create Blog Post

The system shall allow students to create a blog post.

Acceptance criteria:

1. The post must have a title.
2. The post must have content.
3. The post may have a category or activity type.
4. The post may have one featured image.
5. The post must be associated with the logged-in student.
6. The post starts as draft or submitted depending on the action.

FR-06: Upload Featured Image

The system shall allow students to upload one featured image.

Acceptance criteria:

1. Accepted file types are .jpg, .jpeg, .png, and .webp.
2. File size should be limited to 5 MB for MVP.
3. The image is uploaded to Supabase Storage.
4. The image path is saved in the database.
5. The image is not shown publicly unless the post is approved and published.

FR-07: Photo and Publication Consent

The system shall require consent each time a post is submitted.

Acceptance criteria:

1. The student must confirm they have permission to use the image.
2. The student must confirm they understand the post may be published publicly after CIS approval.
3. Submission is blocked if consent checkboxes are not selected.
4. Consent timestamp and version are stored.

FR-08: Submit Post for Review

The system shall allow students to submit posts for CIS Staff review.

Acceptance criteria:

1. The post status changes to submitted.
2. The submission timestamp is stored.
3. The post appears in the admin dashboard.
4. The post remains private.

FR-09: Admin Dashboard

The system shall provide a CIS Staff admin dashboard.

Acceptance criteria:

1. Only users with role admin can access admin pages.
2. Admins can see submitted posts.
3. Admins can open individual posts for review.
4. Students cannot access admin pages.

FR-10: Review Post

The system shall allow admins to review submitted posts.

Acceptance criteria:

1. Admin can view title, content, image, author, and consent status.
2. Admin can add an internal note.
3. Admin can change the review status.
4. Review action is recorded in status history if implemented.

FR-11: Request Revision

The system shall allow admins to request revisions.

Acceptance criteria:

1. Admin can mark post as revision_requested.
2. Admin can write a note explaining what needs to change.
3. Student can see the revision request.
4. Student can edit and resubmit the post.

FR-12: Approve Post

The system shall allow admins to approve a post.

Acceptance criteria:

1. Admin can set review_status = approved.
2. The approval timestamp is saved.
3. The approving admin ID is saved.
4. The post is still not public unless is_published = true.

FR-13: Publish Post

The system shall allow admins to publish an approved post.

Acceptance criteria:

1. Only approved posts can be published.
2. Publishing sets is_published = true.
3. Publishing stores published_at.
4. The post appears on the public blog list.
5. The post detail page becomes publicly accessible.

FR-14: Reject Post

The system shall allow admins to reject a post.

Acceptance criteria:

1. Admin can set review_status = rejected.
2. The rejection timestamp is saved.
3. Rejected posts are not public.
4. Rejected posts are retained for one week according to policy.
5. Automatic deletion may be future enhancement; manual cleanup is acceptable for MVP.

FR-15: Archive Post

The system shall allow admins to archive a post.

Acceptance criteria:

1. Admin can set review_status = archived OR is_published = false.
2. Archived posts are not public.

3. Archived posts remain visible to admins.

FR-16: Public Blog List

The system shall provide a public blog list.

Acceptance criteria:

1. Public visitors can view approved and published posts.
2. Pending, draft, rejected, archived, and revision-requested posts do not appear.
3. The page works on desktop and mobile.

FR-17: Public Blog Detail

The system shall provide a public detail page for each published post.

Acceptance criteria:

1. Public users can open a published post.
2. The page shows title, content, image, author display name if approved by CIS policy, and publication date.
3. Unpublished posts return a not-found or access-denied state.

FR-18: Student Edits After Submission

The system shall allow students to edit their own posts.

Acceptance criteria:

1. Students can edit their own posts.
2. Edited content must go through staff approval again before becoming public.
3. In the MVP, editing a published post may temporarily unpublish it until re-approved.
4. Full version history may be considered a future enhancement.

FR-19: Deletion Request

The system should allow students to request deletion of their own published posts.

Acceptance criteria:

1. Student can request deletion.
2. Admin can review the request.
3. Admin can archive or delete the post.

4. The deletion request is recorded.

This feature is desirable but may be simplified in MVP by providing CIS contact instructions if time is limited.

FR-20: Documentation

The system shall include documentation.

Acceptance criteria:

1. README explains project purpose and local setup.
2. Deployment guide explains Vercel and Supabase deployment.
3. Admin guide explains CIS Staff workflow.
4. Maintenance guide explains handoff to future Student Assistants.
5. Database schema is documented.
6. Known limitations and future enhancements are listed.

15. Non-Functional Requirements

15.1 Security

The application must avoid direct password storage. Passwords will be handled by Supabase Auth. Supabase Auth uses bcrypt hashing and randomly generated salt for stored password hashes.

The system must also:

1. Protect admin routes.
2. Restrict database access using RLS.
3. Prevent students from accessing other students' unpublished posts.
4. Validate file types and file sizes.
5. Store secret keys in environment variables.
6. Avoid committing .env files to GitHub.
7. Prevent public access to unapproved content.
8. Sanitize or safely render user-generated content.
9. Avoid raw HTML rendering in the MVP.

15.2 Privacy

The system must collect privacy consent during signup and content/photo consent during every submission.

Student information, including names, emails, blog text, and uploaded images, should be treated carefully. The U.S. Department of Education notes that photos or videos can be education records under FERPA when they are directly related to a student and maintained by an educational agency or institution or a party acting for the institution.

Therefore, the system will include consent language and CIS Staff review before publication.

15.3 Accessibility

The application should follow basic web accessibility practices:

1. Semantic HTML.
2. Form labels.
3. Keyboard-accessible forms and buttons.
4. Sufficient color contrast.
5. Alt text for uploaded images.
6. Responsive layout.
7. Clear error messages.
8. Logical heading structure.

ASU Web Services states that it works on user interfaces and accessibility requirements for supported systems.

15.4 Maintainability

The project must be understandable by a future Student Assistant.

Maintainability requirements:

1. Clear folder structure.
2. Consistent naming conventions.
3. TypeScript types.
4. Comments for complex logic.
5. README and documentation.
6. Database migrations.
7. Environment variable examples.
8. GitHub issues or task board if possible.

15.5 Usability

The application should be simple for students and staff.

Student interface should:

1. Use clear labels.
2. Avoid technical language.
3. Show status clearly.
4. Explain consent clearly.
5. Provide helpful error messages.

Admin interface should:

1. Show pending posts clearly.
2. Make approve/reject/publish actions easy to understand.
3. Prevent accidental publishing.
4. Show confirmation before destructive actions.

15.6 Performance

The application will be small-scale. Expected student users are 10 or fewer per semester. Therefore, performance requirements are moderate.

The system should:

1. Load public pages quickly.
2. Avoid unnecessary database queries.
3. Use pagination or simple limits if post count grows.
4. Compress or restrict uploaded image size.
5. Avoid overly large images.

15.7 Reliability

The system should not lose student submissions.

Reliability requirements:

1. Store posts in the database.
2. Store images in Supabase Storage.
3. Keep code in GitHub.
4. Use database migrations.
5. Recommend production backup plan.
6. Document restore steps.

16. Privacy and Consent Plan

16.1 Consent at Registration

Students will accept privacy and platform-use consent during account creation.

Suggested consent text:

I understand that my name, Angelo State University email address, submitted blog posts, uploaded images, and submission history may be stored and reviewed by CIS Staff for the purpose of operating the ELLI Student Blogging Platform.

Required database fields:

1. privacy_consent_accepted
2. privacy_consent_at
3. privacy_consent_version

16.2 Consent at Each Submission

Each time a student submits a blog post, the student must confirm content and photo-use consent.

Suggested submission consent text:

I confirm that I have the right to submit this blog post and any uploaded image. I understand that, if approved by CIS Staff, this content may be published publicly on the ELLI website or related ASU/CIS promotional pages.

Required database fields:

1. photo_consent_accepted
2. publication_consent_accepted
3. submission_consent_at
4. submission_consent_version

16.3 Photo Consent

Because uploaded images may include students or other individuals, the upload form should include a specific image-use confirmation.

Suggested text:

I confirm that I have permission to upload and publish this image, and that the image does not contain private information, IDs, documents, addresses, phone numbers, or individuals who have not consented to public posting.

16.4 Prohibited Content

Students should be instructed not to upload:

1. Student IDs.
2. Passports.
3. Driver's licenses.
4. Addresses.
5. Phone numbers.
6. Financial information.
7. Medical information.
8. Immigration documents.
9. Private messages.
10. Images of people who have not agreed to public posting.
11. Images involving minors unless CIS has separate approval.
12. Offensive, discriminatory, or inappropriate content.

16.5 Graduation Policy

Published posts may remain publicly available after a student graduates or leaves ASU because students provided consent during signup and submission.

Suggested text:

I understand that approved and published blog posts may remain publicly available after I leave or graduate from Angelo State University, unless I request removal.

16.6 Deletion Request Policy

Students may request deletion of their published posts.

Suggested public page text:

To request removal of a published post, please contact CIS or submit a deletion request through your student dashboard.

MVP implementation options:

1. Implement a deletion request form.

2. Or provide CIS contact instructions and document the process.

16.7 Legal and Policy Review

This project is a technical prototype. Final legal, FERPA, privacy, media release, and institutional policy language should be reviewed by CIS and appropriate ASU offices before production use.

17. Content Review and Publishing Workflow

The platform will use a structured workflow to ensure that student submissions are not publicly visible until reviewed and published by CIS Staff.

17.1 Status Definitions

Status	Meaning	Publicly Visible
draft	Student is still writing	No
submitted	Student submitted for review	No
under_review	CIS Staff is reviewing	No
revision_requested	CIS Staff requested changes	No
approved	CIS Staff approved content	Not automatically
published	Content is approved and visible	Yes
rejected	CIS Staff rejected the post	No
archived	Post is hidden but retained	No

For implementation, the system may use:

```
review_status = approved
AND
is_published = true
```

as the final public visibility rule.

17.2 Standard Workflow

1. Student creates a draft.
2. Student uploads featured image.

3. Student accepts photo and publication consent.
4. Student submits the post.
5. Post becomes submitted.
6. CIS Staff reviews the post.
7. CIS Staff may request revision, reject, approve, or publish.
8. If revision is requested, student edits and resubmits.
9. If approved, admin may publish.
10. Once approved and published, the post appears publicly.

17.3 Revision Workflow

If ELLI Coordinator or CIS Staff requests changes:

1. Admin sets status to revision_requested.
2. Admin writes a note explaining changes needed.
3. Student sees the note.
4. Student edits the post.
5. Student resubmits.
6. Post returns to submitted.
7. Admin reviews again.

17.4 Editing After Publication

Students may edit their posts after submission or publication. However, edits will not automatically appear publicly.

MVP policy:

1. Student edits a published post.
2. The post becomes unpublished or returns to review status.
3. CIS Staff reviews the edited content.
4. CIS Staff approves and publishes again.
5. The updated version becomes public.

Future enhancement:

A later version may keep the previous approved version public while a new revision is under review. This would require a separate post versioning system.

17.5 Rejected Post Retention

Rejected posts will be kept for one week. After one week, they may be deleted manually by CIS Staff.

MVP approach:

1. Store rejected_at.
2. Show rejected posts in admin dashboard.
3. Document that CIS Staff should delete rejected posts after one week.

Future enhancement:

1. Add scheduled automatic cleanup.
2. Send reminders to admins.

18. Data Retention and Deletion Policy

18.1 Published Posts

Published posts may remain publicly available after the student graduates or leaves ASU, unless the student requests removal.

18.2 Rejected Posts

Rejected posts will be retained for one week after rejection. After one week, they may be deleted.

18.3 Draft Posts

Draft retention may be flexible. For the MVP, drafts can remain unless deleted by the student or admin.

18.4 Archived Posts

Archived posts are not publicly visible but remain available to admins.

18.5 Image Deletion

Uploaded images may be deleted or replaced, but changes to a published post's image must go through staff approval and publishing again before becoming public.

18.6 Deletion Requests

If a student requests deletion:

1. CIS Staff reviews the request.
2. CIS Staff may archive or delete the post.
3. The action should be recorded.
4. Associated images should also be removed or archived according to CIS policy.

19. Backup and Recovery Plan

19.1 Prototype Backup

During Summer I development, the backup plan will include:

1. GitHub repository for code.
2. SQL migration files for database schema.
3. .env.example file without real secrets.
4. README setup instructions.
5. Sample seed data for demo.
6. Documentation folder in the repository.

No real sensitive student data should be used in early prototype testing.

19.2 Production Backup Recommendation

For production, the recommended backup plan is:

1. Use Supabase Pro daily backups if cloud hosting is used.
2. Keep database migrations in GitHub.
3. Export published posts and metadata at the end of each semester.
4. Export image storage at the end of each semester if feasible.
5. Maintain a restore guide for future Student Assistants.
6. Test the restore process at least once before production launch.

Supabase documentation explains that restoration requires selecting a backup, confirming the restore, and planning for downtime because the project is inaccessible during restoration.

19.3 Point-in-Time Recovery

Point-in-Time Recovery, PITR, is not necessary for the Summer I MVP. It may be considered later if CIS requires more advanced recovery. Supabase documentation states that PITR can restore to a chosen point with up to seconds-level granularity, but it is a Pro/Team/Enterprise add-on and has additional cost. For this project's small expected usage, daily backups plus semester exports should be sufficient for an initial production plan.

20. Proposed Database Design

20.1 *profiles* Table

Purpose: Store public application-level user profile information.

Column	Type	Description
id	uuid	References Supabase Auth user ID
email	text	User email
full_name	text	User full name
role	text	<i>student</i> or <i>admin</i>
privacy_consent_accepted	boolean	Whether signup consent was accepted
privacy_consent_at	timestamp	Consent timestamp
privacy_consent_version	text	Consent version
created_at	timestamp	Account creation date
updated_at	timestamp	Last profile update

Rules:

1. Students can view and update limited parts of their own profile.
2. Admins can view all profiles.
3. Only admins or service role can change user role.

20.2 *posts* Table

Purpose: Store blog post content and review/publishing status.

Column	Type	Description
id	uuid	Post ID
author_id	uuid	References <i>profiles.id</i>

Column	Type	Description
title	text	Post title
slug	text	URL-friendly identifier
content	text	Main blog content
category	text	Optional activity/category
featured_image_path	text	Storage path
image_alt_text	text	Accessibility text
review_status	text	Draft/submitted/revision/approved/rejected/archived
is_published	boolean	Public visibility
admin_note	text	Staff note for revisions/rejections
photo_consent_accepted	boolean	Photo-use consent
publication_consent_accepted	boolean	Public posting consent
submission_consent_at	timestamp	Submission consent time
submission_consent_version	text	Consent version
submitted_at	timestamp	Submission time
approved_at	timestamp	Approval time
approved_by	uuid	Admin who approved
published_at	timestamp	Publication time
published_by	uuid	Admin who published
rejected_at	timestamp	Rejection time
archived_at	timestamp	Archive time
created_at	timestamp	Creation time

Rules:

1. Students can create posts for themselves.
2. Students can view and edit their own posts.
3. Students cannot approve or publish posts.
4. Admins can view and manage all posts.
5. Public users can only view posts where *review_status* = *approved* and *is_published* = *true*.

20.3 *post_status_history* Table

Purpose: Track status changes for accountability.

Column	Type	Description
id	uuid	History record ID
post_id	uuid	Related post
changed_by	uuid	User who changed status
previous_status	text	Old status
new_status	text	New status
previous_is_published	boolean	Old publication state
new_is_published	boolean	New publication state
note	text	Optional note
created_at	timestamp	Change timestamp

This table is recommended but may be simplified if time is limited.

20.4 *deletion_requests* Table

Purpose: Allow students to request removal of published posts.

Column	Type	Description
id	uuid	History record ID
post_id	uuid	Related post

Column	Type	Description
requester_id	uuid	Student requesting deletion
reason	text	Reason for request
status	text	pending, approved, denied, completed
admin_note	text	Admin response
resolved_by	uuid	Admin who resolved
created_at	timestamp	Request date
resolved_at	timestamp	Resolution date

This feature is useful but may be implemented after the core workflow if time is limited.

21. Storage Design

21.1 Storage Bucket

Recommended bucket name:

post-images

21.2 File Path Convention

Recommended file path format:

post-images/{user_id}/{post_id}/featured-{timestamp}.{extension}

Example:

post-images/USER_UUID/POST_UUID/featured-20260610.png

21.3 Image Access Policy

Preferred production approach:

1. Store images in a private bucket.

2. Allow students to access their own uploaded images.
3. Allow admins to access all images.
4. Allow public display only for approved and published posts.

Simplified MVP approach:

1. Store images with hard-to-guess file paths.
2. Do not expose image URLs until a post is published.
3. Document that stricter private-bucket policies should be reviewed before production.

21.4 Image Requirements

For MVP:

1. One featured image per post.
2. File size limit: 5 MB.
3. Supported formats: .jpg, .jpeg, .png, .webp.
4. Student must provide alt text or a short image description.
5. Student must confirm image-use consent.

22. Security Plan

22.1 Password Security

The application will not store passwords in the application database.

Authentication will be handled by Supabase Auth. Supabase Auth uses bcrypt password hashing and random salt.

The profiles table will store user profile and role information, but not passwords.

22.2 Email Domain Restriction

Only @angelo.edu email addresses will be allowed.

Implementation plan:

1. Client-side validation on signup form.
2. Server-side validation before creating profile.
3. Supabase Auth hook or server route validation if feasible.

Supabase Auth Hooks can inspect a user before creation and reject requests, including enforcing custom signup policies such as requiring users to belong to a specific email domain.

22.3 Role-Based Access

Roles:

1. student
2. admin

Role rules:

1. New users default to student.
2. Admin role is assigned manually by developer/CIS.
3. Students cannot assign themselves admin role.
4. Admin pages check role before rendering.
5. Database policies also enforce access.

22.4 Row Level Security

RLS policies should enforce:

1. Students can read their own profile.
2. Students can read, create, and update their own posts.
3. Students cannot set `is_published = true`.
4. Students cannot approve posts.
5. Admins can read and update all posts.
6. Public users can read only published posts.
7. Public users cannot read pending or rejected posts.

22.5 Environment Variables

Secrets must be stored in environment variables.

Examples:

```
NEXT_PUBLIC_SUPABASE_URL=  
NEXT_PUBLIC_SUPABASE_ANON_KEY=  
SUPABASE_SERVICE_ROLE_KEY=
```

Rules:

1. Never commit `.env.local`.
2. Commit `.env.example` only.
3. Use Vercel environment variables for deployment.
4. Service role key must never be exposed to the browser.

22.6 Input Validation

The system should validate:

1. Required fields.
2. Email domain.
3. Title length.
4. Content length.
5. File type.
6. File size.
7. Consent checkboxes.
8. Admin status transitions.

22.7 XSS Prevention

Because the platform stores user-generated text, the MVP should avoid raw HTML input.

Recommended approach:

1. Use plain text or Markdown-like formatting.
2. Escape rendered content.
3. Avoid *dangerouslySetInnerHTML*.
4. If rich text is added later, sanitize HTML carefully.

23. Accessibility Plan

The application should follow basic accessibility principles.

23.1 Forms

1. Every input should have a label.
2. Required fields should be clearly marked.
3. Error messages should explain the issue.
4. Consent checkboxes should have clear text.

23.2 Images

1. Students should provide alt text.
2. Admins should review alt text before publication.
3. Decorative images should be handled appropriately.

23.3 Navigation

1. Keyboard navigation should work.
2. Buttons should be focusable.
3. Links should have meaningful text.
4. Admin actions should be clearly labeled.

23.4 Responsive Design

The app should work on:

1. Desktop computers.
2. Laptops.
3. Tablets.
4. Mobile phones.

Students are expected to mainly write posts on computers, but the site should be accessible on mobile devices.

24. User Interface Plan

24.1 Public Pages

Home Page

Purpose:

Introduce ELLI and link to blog posts.

Sections:

1. ELLI introduction.
2. Short explanation of student experiences.
3. Button to view blog posts.
4. Button for students to log in.
5. Optional featured posts section.

Blog List Page

Purpose:

Show approved and published posts.

Content:

1. Post title.
2. Featured image.
3. Short excerpt.
4. Author display name if approved.
5. Publication date.
6. Link to full post.

Blog Detail Page

Purpose:

Show full approved post.

Content:

1. Title.
2. Featured image.
3. Image alt text.
4. Content.
5. Author display name.
6. Publication date.
7. Optional removal request contact text.

24.2 Student Pages

Signup Page

Fields:

1. Full name.
2. @angelo.edu email.
3. Password.
4. Confirm password.
5. Privacy consent checkbox.

Login Page

Fields:

1. Email.
2. Password.
3. Login button.

4. Password reset link if implemented.

Student Dashboard

Content:

1. Welcome message.
2. Create new post button.
3. List of student's posts.
4. Status badge for each post.
5. Edit button.
6. View admin note if revision requested.

Create/Edit Post Page

Fields:

1. Title.
2. Category/activity.
3. Content.
4. Featured image upload.
5. Image alt text.
6. Photo-use consent checkbox.
7. Publication consent checkbox.
8. Save draft button.
9. Submit for review button.

24.3 Admin Pages

Admin Dashboard

Content:

1. List of submitted posts.
2. Filter by status if time allows.
3. Author name.
4. Submitted date.
5. Status.
6. Review button.

Admin Review Page

Content:

1. Student name.

2. Student email.
3. Post title.
4. Content.
5. Featured image.
6. Consent indicators.
7. Admin note field.
8. Buttons:
 - Mark under review
 - Request revision
 - Approve
 - Publish
 - Reject
 - Archive

Admin action buttons should include confirmation prompts for publishing, rejection, archive, and deletion.

25. Development Methodology

The project will use an incremental development approach.

Each week will produce a testable milestone. The goal is to avoid waiting until the end to integrate all features.

Development principles:

1. Build core workflow first.
2. Keep the MVP small and complete.
3. Commit code frequently.
4. Document decisions as they are made.
5. Test each role separately.
6. Avoid adding future features before MVP completion.
7. Keep UI simple until core functionality works.
8. Use sample data before real data.
9. Keep deployment working throughout the project.

Daily development routine:

1. Review project board.
2. Choose 2–4 tasks for the day.
3. Implement one feature at a time.
4. Test as student and admin.
5. Commit changes.
6. Update documentation if behavior changes.

7. Record blockers or questions.

26. Detailed Development Timeline

June 2, 2026 – July 3, 2026

The development period is approximately one month. The timeline below assumes consistent daily work, with the understanding that some days may require adjustment based on feedback, bugs, or academic requirements.

Phase 1: Proposal, Planning, and Project Setup

June 2 – June 7

Goal:

Finalize project scope, create development environment, set up repository, configure Next.js, Supabase, Tailwind CSS, and basic authentication structure.

Phase 2: Student Blog Submission Workflow

June 8 – June 14

Goal:

Build the student-facing functionality for creating, editing, saving, submitting, and viewing blog posts.

Phase 3: CIS Staff Admin Review and Public Publishing

June 15 – June 21

Goal:

Build admin workflow and public blog display.

Phase 4: Policy Features, Security, Accessibility, and Documentation

June 22 – June 27

Goal:

Improve security, data handling, responsive design, accessibility, backup planning, and documentation.

Phase 5: Final Polish, Report, Demo, and Submission

June 28 – July 3

Goal:

Finish the final version, polish UI, prepare final report, prepare presentation/demo, and complete submission.

27. Final Deliverables

By July 3, 2026, the following deliverables should be completed:

27.1 Software Deliverables

1. Working Next.js web application.
2. Student registration and login.
3. Student dashboard.
4. Blog post submission.
5. Featured image upload.
6. Consent collection.
7. Admin dashboard.
8. Admin review workflow.
9. Public blog page.
10. Public post detail page.
11. Role-based access control.
12. Supabase database schema.
13. Supabase Storage setup.

27.2 Documentation Deliverables

1. Project proposal.
2. README.
3. Local setup guide.
4. Supabase setup guide.
5. Vercel deployment guide.
6. Database schema documentation.
7. Admin user guide.
8. Maintenance handoff guide.
9. Backup and restore recommendation.
10. Security checklist.
11. Accessibility checklist.
12. Known limitations.
13. Future enhancement list.

27.3 Academic Deliverables

1. Final report.
2. Demo script.
3. Screenshots.
4. GitHub repository link.
5. Deployment link if available.
6. Summary of lessons learned.
7. Summary of challenges and solutions.

28. Testing Plan

28.1 Student Testing

Test cases:

1. Student can register with @angelo.edu.
2. Student cannot register with non-@angelo.edu.
3. Student can log in.
4. Student can log out.
5. Student can accept privacy consent.
6. Student can create draft.
7. Student can submit post.
8. Student cannot submit without required fields.
9. Student cannot submit without consent.

28.2 Admin Testing

Test cases:

1. Admin can log in.
2. Admin can access admin dashboard.
3. Student cannot access admin dashboard.
4. Admin can view submitted posts.
5. Admin can open post review page.
6. Admin can request revision.
7. Admin can approve.
8. Admin can publish approved post.
9. Admin cannot accidentally publish rejected post.
10. Admin can reject.
11. Admin can archive.
12. Admin note appears to student when revision requested.

28.3 Public Testing

Test cases:

1. Public visitor can view homepage.
2. Public visitor can view blog list.
3. Public visitor can view published post.
4. Public visitor cannot view submitted post.
5. Public visitor cannot view rejected post.
6. Public visitor cannot access dashboard.
7. Public visitor cannot access admin page.

28.4 Security Testing

Test cases:

1. Unauthenticated user cannot access dashboard.
2. Student cannot access admin route.
3. Student cannot publish post.
4. Student cannot update another student's post.
5. Public user cannot query private posts.
6. Environment variables are not exposed.
7. Service role key is not used in browser code.
8. RLS policies are active.

28.5 Accessibility Testing

Test cases:

1. Forms have labels.
2. Buttons are keyboard accessible.
3. Images have alt text.
4. Error messages are readable.
5. Mobile layout works.
6. Heading order is logical.
7. Status badges are understandable.

29. Risk Analysis and Mitigation

Risk 1: Scope Creep

Description:

The project could become too large if features like comments, ASU SSO, multiple image galleries, rich text editing, email notifications, or analytics are added too early.

Mitigation:

Keep Summer I focused on MVP. Move advanced features to future enhancements.

Risk 2: ASU IT Hosting Uncertainty

Description:

ASU IT/Web Services may not immediately approve hosting or integration.

Mitigation:

Develop the project as a prototype first. Document hosting options. Prepare for ASU IT review after MVP completion.

Risk 3: Privacy and FERPA Concerns

Description:

Student names, emails, photos, and experiences may involve privacy considerations.

Mitigation:

Collect consent at signup and submission. Require CIS Staff approval before publication. Avoid publishing unapproved content. Request institutional review before production.

Risk 4: Image Misuse

Description:

Students may upload images containing people who did not consent or private documents.

Mitigation:

Require photo-use consent on every submission. Provide warnings. Admins review images before publication.

Risk 5: Authentication Complexity

Description:

ASU SSO may be difficult to implement without ASU IT.

Mitigation:

Use Supabase Auth for MVP. Restrict signups to @angelo.edu. Treat ASU SSO as future enhancement.

Risk 6: Security Misconfiguration

Description:

Incorrect RLS policies could expose unpublished posts.

Mitigation:

Enable RLS early. Test with student, admin, and public users. Keep public query conditions strict.

Risk 7: Time Constraint

Description:

The project period is only about one month.

Mitigation:

Follow phased schedule. Complete core workflow by June 21. Use final two weeks for testing, documentation, and polish.

Risk 8: Future Maintenance

Description:

After the initial developer leaves, CIS may need to maintain the application.

Mitigation:

Use clear documentation, GitHub repository, environment examples, admin guide, and handoff guide. Transfer repository to CIS or ASU-managed account if possible.

Risk 9: Cost Overruns

Description:

Cloud hosting may exceed the expected cost due to add-ons or overages.

Mitigation:

Use free tier for prototype. Estimate paid cost before production. Set spending limits where possible. Avoid unnecessary paid add-ons.

Risk 10: Data Loss

Description:

Posts or images could be lost due to database or storage issues.

Mitigation:

Use GitHub for code, migrations for schema, Supabase backups for production, and semester exports for published content.

30. Success Criteria

The project will be considered successful if the following are true by July 3, 2026:

1. A student can register using an @angelo.edu email.
2. A student can log in and access their dashboard.

3. A student can create and submit a blog post.
4. A student can upload one featured image.
5. A student must accept required consent before submission.
6. CIS Staff admin can view submitted posts.
7. CIS Staff admin can approve, reject, request revision, and publish posts.
8. Only approved and published posts appear publicly.
9. Students cannot access admin pages.
10. Public users cannot access unpublished posts.
11. The application is responsive.
12. The project is deployed for demonstration.
13. Documentation is complete enough for a future Student Assistant to understand and continue the project.
14. The final report and demo clearly explain the project's purpose, architecture, security, limitations, and future work.

31. Future Enhancements

After the Summer I MVP, future versions may include:

1. ASU SSO integration.
2. ASU IT/Web Services hosting or official ASU domain.
3. Multiple image upload.
4. Image gallery per post.
5. Email notifications for submission, revision, approval, and publication.
6. Full post version history.
7. Previous-public-version preservation during revisions.
8. Automatic rejected-post deletion after one week.
9. Advanced admin filters.
10. Search and categories.
11. Draft autosave.
12. Rich text editor with safe sanitization.
13. Content export for CIS.
14. Analytics for public posts.
15. Integration with ELLI promotional landing page.
16. Integration with LiveWhale or official ASU web pages if approved.
17. More advanced accessibility testing.
18. Formal privacy/media release workflow.
19. Staff training materials.
20. Student onboarding guide.

32. Conclusion

The ELLI Student Blogging Platform is a practical, real-world web development project that

supports CIS and the ELLI program while also meeting the academic expectations of a Computer Science project course. The project involves full-stack development, authentication, authorization, database design, image upload, administrative workflow, privacy consent, public content publishing, deployment planning, and documentation.

The Summer I MVP will focus on a realistic and complete core system rather than unnecessary extra features. By July 3, 2026, the goal is to deliver a working prototype where ELLI students can submit blog posts and CIS Staff can review, approve, and publish those posts. The platform will be documented so that CIS or future Student Assistants can maintain and improve it.

The final production path will depend on ASU IT/Web Services review, CIS requirements, hosting decisions, privacy considerations, and long-term maintenance planning. The MVP will provide a strong foundation for that future discussion.